

Vertex Cover

Muşuroi David & Tănasă Damian

Facultatea de Automatică şi Calculatoare

Abstract. Această lucrare analizează problema *Vertex Cover*, una dintre problemele clasice NP-Hard din teoria grafurilor. Sunt prezentate definiţia problemei, aplicaţii practice, demonstrarea caracterului NP-Hard şi o reducere de la problema *Independent Set*. De asemenea, sunt descrişi şi evaluaţi trei algoritmi de rezolvare: un algoritm exact bazat pe backtracking şi două soluţii euristice, greedy şi de aproximare.

Keywords: Vertex Cover, NP-Hard, Backtracking, Aproximare, Greedy

1 Introducere

Problema *Vertex Cover* este una dintre cele mai importante probleme de optimizare din teoria grafurilor şi complexitate computaţională. Ea apare natural în securitatea reţelelor, proiectarea circuitelor, bioinformatică şi inteligenţă artificială.

Fiind NP-Hard, problema nu poate fi rezolvată exact în timp polinomial pentru grafuri mari, motiv pentru care sunt necesari algoritmi aproximaţi, euristici şi metode parametrizate.

1.1 Descrierea problemei

Un *Vertex Cover* într-un graf neorientat $G = (V, E)$ este o mulţime de noduri $S \subseteq V$ astfel încât orice muchie $(u, v) \in E$ are cel puţin un capăt în S .

Problema *Vertex Cover minim* constă în determinarea unei mulţimi S de cardinal minim care să satisfacă această proprietate.

1.2 Exemple de aplicaţii practice

Securitatea şi monitorizarea reţelelor

Într-o reţea de calculatoare:

- nodurile reprezintă computere, servere sau routere;
- muchiile reprezintă conexiuni de reţea.

Problema: Alegerea minimului de noduri pe care trebuie instalate sisteme de monitorizare astfel încât fiecare conexiune să fie supravegheată.

Plasarea camerelor de supraveghere Modelarea se face astfel:

- nodurile reprezintă intersecții;
- muchiile reprezintă străzi.

O cameră pusă într-o intersecție vede toate străzile care pleacă din ea.

Problema: Care este numărul minim de intersecții unde trebuie instalate camere astfel încât fiecare stradă să fie monitorizată?

Programarea sarcinilor În sisteme cu conflicte:

- nodurile reprezintă task-uri;
- muchiile reprezintă conflicte între task-uri.

Problema: Selectarea minimului de task-uri care trebuie blocate pentru a elimina conflictele.

2 Demonstrație NP-Hard

Problema *Vertex Cover* aparține clasei NP, deoarece pentru o soluție dată se poate verifica în timp polinomial dacă fiecare muchie este acoperită.

Pentru a demonstra că problema este NP-Hard, vom arăta că *Independent Set* $\leq_p$ *Vertex Cover*.

2.1 Reducere de la o problemă NP-Hard

Fie (G, k) o instanță a problemei *Independent Set*. Construim instanța (G', k') pentru *Vertex Cover* astfel:

$$V' = V, \quad E' = E, \quad k' = |V| - k$$

Există un *Independent Set* de dimensiune k în G dacă și numai dacă $V \setminus S$ este un *Vertex Cover* de dimensiune k' în G' . Această reducere este polinomială, deci problema *Vertex Cover* este NP-Hard.

3 Prezentare algoritmi

3.1 Algoritmul Recursiv Backtracking

Descriere Algoritmul explorează exhaustiv toate posibilitățile de acoperire. Pentru fiecare muchie neacoperită (u, v) se generează două ramuri: una în care este selectat u și una în care este selectat v .

Complexitate Complexitatea în cel mai rău caz este $O(2^N \cdot N^2)$.

Avantaje și dezavantaje

- Garantează găsirea numărului minim absolut de noduri (+)
- Timpul de execuție crește exponențial (inutilizabil pentru grafuri cu mai mult de 50-60 de noduri) (-)
- Consumul de memorie este mare din cauza copierii repetate a grafului pe stivă (-)

3.2 Algoritmul de Aproximare

Descriere Algoritmul selectează iterativ o muchie și adaugă ambii săi capete în soluție, eliminându-le ulterior din graf.

Complexitate Complexitatea este $O(N^3)$.

Avantaje și dezavantaje

- foarte rapid (+)
- soluția este garantat de cel mult două ori mai mare decât optimul (+)
- precizie redusă (-)

3.3 Algoritmul Greedy

Descriere La fiecare pas se alege nodul cu grad maxim, deoarece acesta acoperă cele mai multe muchii simultan.

Complexitate Complexitatea este $O(N^3)$.

Avantaje și dezavantaje

- soluții bune în practică (+)
- fără garanție strictă de aproximare (-)
- mai lent decât algoritmul de aproximare (-)

4 Evaluare**4.1 Construirea setului de teste**

Setul de teste a fost generat folosind un program Python care creează grafuri dense, rare, bipartite și aleatoare, cu dimensiuni între 20 și 1200 de noduri.

4.2 Specificațiile sistemului de calcul

- Sistem de operare: Windows 10 (64-bit);
- Procesor: Intel Core i7 (generația a 10-a);
- Memorie RAM: 8 GB;
- Arhitectură: x64.

4.3 Rezultatele evaluării

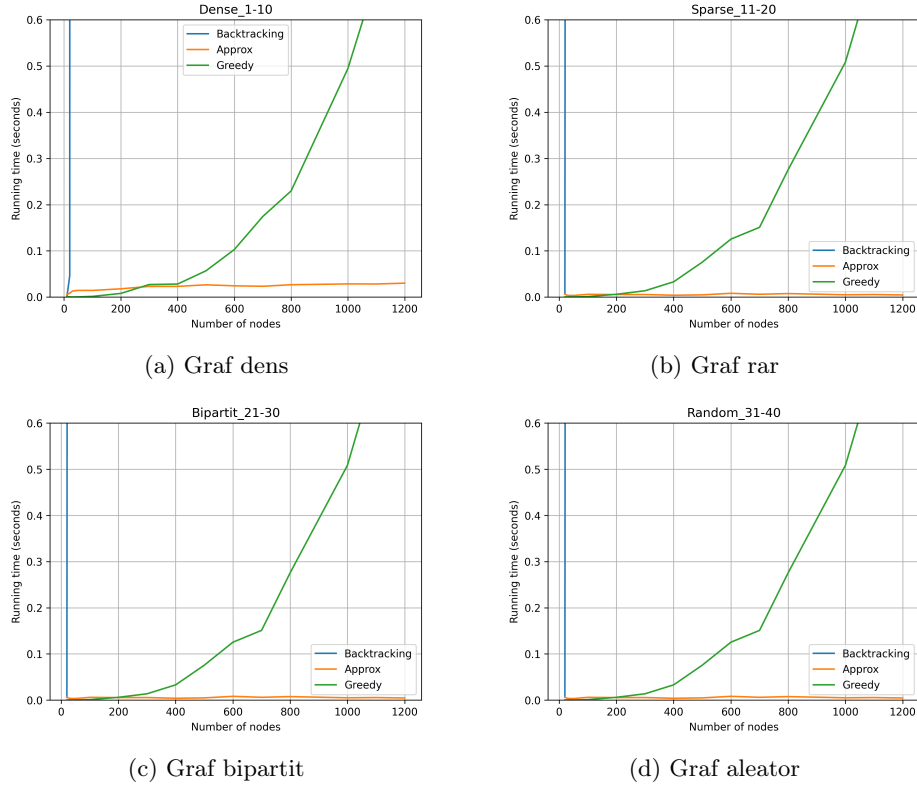


Fig. 1: Rezultatele experimentale pentru algoritmi analizați pe diferite tipuri de grafuri.

Table 1: Acuratețea soluțiilor obținute pentru diferite tipuri de grafuri

TestID	Nodes	Category	Backtracking	Approximation	Greedy
T01_Dense_10	10	Dense	8	10	8
T02_Dense_20	20	Dense	17	20	17
T03_Dense_30	30	Dense	27	30	27
T16_Sparse_10	10	Sparse	4	8	4
T17_Sparse_20	20	Sparse	11	16	12
T18_Sparse_30	30	Sparse	19	28	19
T31_Bipartit_10	10	Bipartit	5	8	5
T32_Bipartit_20	20	Bipartit	10	18	10
T33_Bipartit_30	30	Bipartit	15	30	16
T46_Random_10	10	Random	7	8	7
T47_Random_20	20	Random	14	18	15
T48_Random_30	30	Random	24	28	24

4.4 Interpretarea rezultatelor

- **Algoritmul de aproximare** este cel mai eficient din punct de vedere al timpului de calcul. Acesta prezintă o creștere lentă a timpului de execuție chiar și pentru valori mari ale numărului de noduri, datorită strategiei euristice utilizate și evitării explorării exhaustive a spațiului soluțiilor.
- **Algoritmul greedy** se situează pe locul al doilea ca performanță. El realizează alegeri locale optime la fiecare pas, ceea ce îi permite să ruleze relativ rapid. Deși este mai lent decât algoritmul de aproximare, complexitatea sa rămâne acceptabilă pentru instanțe de dimensiuni medii.
- **Algoritmul de backtracking** prezintă o creștere exponențială a timpului de execuție odată cu mărirea numărului de noduri. Acest comportament este cauzat de explorarea unui număr foarte mare de combinații posibile pentru a garanta soluția optimă. În consecință, algoritmul devine impracticabil pentru instanțe mai mari de aproximativ 30 de noduri.

Din punctul de vedere al acurateței soluției:

- Analizând calitatea soluțiilor obținute, **algoritmul de backtracking** oferă cele mai bune rezultate, deoarece garantează găsirea soluției optime. El evaluează toate posibilitățile și, prin urmare, nu ratează nicio soluție mai bună.
- **Algoritmul greedy** produce soluții foarte apropiate de cea optimă în majoritatea cazurilor. Deși nu garantează optimalitatea, strategia sa bazată pe alegeri locale bune conduce, de regulă, la rezultate satisfăcătoare, mai ales pentru instanțe de dimensiuni medii.
- În schimb, **algoritmul de aproximare** are cea mai scăzută acuratețe. Deși este foarte rapid, el sacrifică din calitatea soluției pentru a obține un timp de execuție redus. Soluțiile furnizate pot fi semnificativ mai îndepărtate de optim față de cele obținute prin greedy sau backtracking, mai ales pentru probleme mai complexe.

5 Concluzii

În urma analizei experimentale se poate afirma că toate cele trei soluții au atât avantaje, cât și dezavantaje, însă fiecare este potrivită pentru contexte diferite, astfel încât să se obțină un randament optim.

Algoritmul backtracking oferă cea mai bună soluție din punct de vedere al acurateței, însă este foarte costisitor din punct de vedere al timpului de execuție. Acesta poate fi utilizat atunci când:

- dorim să determinăm un vertex cover pentru grafuri cu cel mult 30 de noduri
- graful nu se modifică și algoritmul trebuie rulat o singură dată.

Algoritmul de aproximare este cel mai eficient din punct de vedere al timpului de execuție, însă nu garantează obținerea soluției optime. El este potrivit atunci când:

- dorim să găsim rapid un vertex cover (nu neapărat cel mai bun) pentru grafuri mari
- grafurile se modifică frecvent.

Algoritmul greedy este mai rapid decât backtracking-ul, dar mai lent decât algoritmul de aproximare, oferind însă o soluție suficient de apropiată de cea optimă. Acesta poate fi folosit pentru grafuri cu un număr mediu de noduri atunci când:

- nu este strict necesară o soluție perfectă
- nu avem nevoie de obținerea acesteia în cel mai scurt timp posibil.

Astfel, **greedy** reprezintă o metodă care echilibrează eficient performanța de timp cu acuratețea soluției.

6 Bibliografie

References

1. Wikipedia: Vertex Cover.
2. GeeksforGeeks: Vertex Cover Problem.
3. StackOverflow: Greedy algorithms for Vertex Cover.
4. Curs Analiza Algoritmilor (AA).